

# ENTERPRISE SYSTEM ANALYSIS

## Brand Depot Management & Field Operation Tracking System

---

*Full SDLC Analysis & Architecture Design*

Version 1.0 | Confidential

Document Date: May 7, 2026

# Table of Contents

Table of Contents ..... 2

1. Executive Summary ..... 5

    1.1 Business Problem Statement ..... 5

    1.2 Proposed Solution ..... 5

    1.3 Technology Stack Overview ..... 5

2. Business Analysis ..... 7

    2.1 Business Domain Analysis ..... 7

    2.2 Key Business Entities ..... 7

    2.3 Business Goals & KPIs..... 7

3. SDLC Phase Analysis ..... 8

    3.1 Phase Overview ..... 8

    3.2 Agile Sprint Plan (High Level) ..... 8

4. Functional Requirements ..... 9

    4.1 Authentication & Authorization Module ..... 9

    4.2 User Management Module ..... 9

    4.3 Brand Depot Management Module ..... 9

    4.4 Product Management Module ..... 10

    4.5 Visit Tracking Module ..... 10

    4.6 Reporting & Analytics Module..... 11

5. Non-Functional Requirements..... 12

6. User Roles & Permissions ..... 13

    6.1 Role Definitions ..... 13

    6.2 Permission Matrix..... 13

7. User Stories ..... 14

    7.1 Field Handler Stories ..... 14

    7.2 Manager Stories ..... 14

    7.3 Admin Stories ..... 14

8. Database Design ..... 16

    8.1 Database Architecture Overview ..... 16

    8.2 Core Entity Relationship Description ..... 16

    8.3 PostgreSQL Schema..... 16

        Table: users ..... 16

        Table: brand\_depots ..... 17

        Table: products ..... 17

        Table: visits..... 18

|                                                                      |    |
|----------------------------------------------------------------------|----|
| Tables: assignments, visit_images, activity_logs, notifications..... | 19 |
| 9. Backend Architecture .....                                        | 21 |
| 9.1 Architecture Pattern: Modular Monolith → Microservices.....      | 21 |
| 9.2 Folder Structure.....                                            | 21 |
| 9.3 API Versioning Strategy .....                                    | 21 |
| 9.4 Middleware Stack .....                                           | 22 |
| 9.5 Centralized Response Format .....                                | 22 |
| 10. REST API Design .....                                            | 23 |
| 10.1 Authentication Endpoints .....                                  | 23 |
| 10.2 Depot Endpoints.....                                            | 23 |
| 10.3 Visit Endpoints.....                                            | 23 |
| 10.4 Reports Endpoints.....                                          | 24 |
| 11. Frontend Architecture.....                                       | 25 |
| 11.1 React.js Project Structure .....                                | 25 |
| 11.2 State Management Strategy .....                                 | 25 |
| 11.3 Key Frontend Libraries .....                                    | 25 |
| 12. Mobile Architecture .....                                        | 27 |
| 12.1 Framework Selection .....                                       | 27 |
| 12.2 Offline Sync Strategy .....                                     | 27 |
| 13. Security Architecture .....                                      | 28 |
| 13.1 JWT Authentication Flow .....                                   | 28 |
| 13.2 Security Controls .....                                         | 28 |
| 13.3 Password Security.....                                          | 29 |
| 14. DevOps & Deployment Architecture.....                            | 30 |
| 14.1 Docker Compose (Development).....                               | 30 |
| 14.2 GitHub Actions CI/CD Pipeline .....                             | 30 |
| 14.3 Infrastructure Architecture .....                               | 31 |
| 15. Real-Time & Queue Architecture.....                              | 32 |
| 15.1 Socket.IO Events.....                                           | 32 |
| 15.2 Queue Job Types .....                                           | 32 |
| 16. Caching Strategy .....                                           | 33 |
| 16.1 Redis Cache Layers .....                                        | 33 |
| 17. Testing Strategy.....                                            | 34 |
| 17.1 Testing Pyramid .....                                           | 34 |
| 18. Scalability Strategy .....                                       | 35 |
| 18.1 Horizontal Scaling Approach .....                               | 35 |

18.2 Database Scaling ..... 35

18.3 Modular Monolith to Microservices Evolution ..... 35

19. Reporting & Analytics..... 36

19.1 Built-in Reports..... 36

20. Risk Analysis ..... 37

21. Future Enhancement Plan ..... 38

21.1 Short-Term (0–6 Months Post-Launch)..... 38

21.2 Medium-Term (6–18 Months) ..... 38

21.3 Long-Term (18+ Months)..... 38

22. Final Recommendations ..... 39

22.1 Development Priority Matrix..... 39

# 1. Executive Summary

The Brand Depot Management & Field Operation Tracking System (BDMFOTS) is an enterprise-grade digital platform designed to modernize, streamline, and bring full accountability to brand distribution operations. This document presents a comprehensive SDLC analysis encompassing business requirements, architecture decisions, database design, API specifications, security protocols, and deployment strategies.

The system addresses a critical operational gap faced by brand distribution companies: the lack of real-time visibility into field activities, product availability at distribution points (depots), and staff performance accountability. BDMFOTS delivers an integrated web application, mobile operations platform, and administrative backend that together form a complete operational intelligence system.

## 1.1 Business Problem Statement

- Field handlers visit brand depots with no centralized tracking or accountability mechanism
- Product availability and shelf conditions are reported manually with delays and inaccuracies
- Managers lack real-time visibility into field operations and cannot monitor staff performance
- Visit records, photos, and reports exist in disparate systems or paper documents
- No GPS verification of depot visits, enabling false reporting
- Analytics and performance metrics require significant manual effort to compile

## 1.2 Proposed Solution

BDMFOTS is a multi-tier, cloud-native enterprise platform with the following core capabilities:

- Real-time GPS-verified field visit tracking with photo documentation
- Centralized depot and product inventory management
- Role-based access control with full audit trail
- AI-assisted analytics dashboards and automated reporting
- Offline-capable mobile application with background synchronization
- Event-driven notification and alert architecture

## 1.3 Technology Stack Overview

| Layer       | Technology              | Purpose                                  |
|-------------|-------------------------|------------------------------------------|
| Backend API | Node.js + Express.js    | RESTful API server, business logic       |
| Database    | PostgreSQL + Prisma ORM | Primary data store, relational integrity |
| Cache       | Redis                   | Session storage, caching, pub/sub        |
| Real-time   | Socket.IO               | Live dashboards, notifications           |

| Layer          | Technology               | Purpose                             |
|----------------|--------------------------|-------------------------------------|
| Queue          | RabbitMQ / Kafka         | Async job processing, event streams |
| Frontend       | React.js + TailwindCSS   | Responsive web application          |
| Mobile         | React Native / Flutter   | Cross-platform mobile app           |
| Storage        | AWS S3 / Cloudinary      | Image and file storage              |
| DevOps         | Docker + GitHub Actions  | Containerization, CI/CD             |
| Infrastructure | AWS / GCP / DigitalOcean | Cloud hosting, scalability          |

## 2. Business Analysis

### 2.1 Business Domain Analysis

Brand distribution businesses operate through a network of geographically distributed depot locations where brand products are stored, displayed, and sold to end customers. Field handlers—the operational backbone—travel to these depots to assess stock levels, verify product placement and shelf conditions, conduct promotions, resolve operational issues, and capture business intelligence.

### 2.2 Key Business Entities

| Entity        | Description                                               | Business Importance       |
|---------------|-----------------------------------------------------------|---------------------------|
| Brand Depot   | Physical or virtual distribution point for brand products | Core operational unit     |
| Product       | Brand-owned product SKU with attributes and status        | Revenue-generating asset  |
| Field Handler | Staff responsible for visiting and managing depots        | Primary operational actor |
| Visit         | Documented interaction between handler and depot          | Key accountability event  |
| Manager       | Supervisory role overseeing handlers and depots           | Strategic oversight       |
| Report        | Aggregated operational analytics at multiple periods      | Business intelligence     |
| Assignment    | Formal allocation of handler responsibility to depot      | Accountability record     |

### 2.3 Business Goals & KPIs

- Reduce unverified field visits to < 2% through GPS geofencing enforcement
- Improve reporting cycle time from weekly manual to daily automated
- Achieve > 95% depot visit compliance rate tracked digitally
- Reduce product stock-out incidents by 30% through real-time monitoring
- Enable managers to view live field status with < 5 second latency
- Generate automated monthly performance reports for all handlers

### 3. SDLC Phase Analysis

#### 3.1 Phase Overview

| Phase                    | Key Activities                                          | Deliverables                         | Duration |
|--------------------------|---------------------------------------------------------|--------------------------------------|----------|
| 1. Planning              | Scope definition, feasibility, resource planning        | Project charter, risk register       | 2 weeks  |
| 2. Requirements          | Stakeholder interviews, use case analysis, user stories | SRS document, use case diagrams      | 3 weeks  |
| 3. System Design         | Architecture, DB design, API contracts, UI wireframes   | HLD, LLD, ERD, API spec              | 4 weeks  |
| 4. Development           | Backend, frontend, mobile coding, unit testing          | Sprint deliverables, code artifacts  | 16 weeks |
| 5. Integration & Testing | Integration tests, QA, UAT, performance testing         | Test reports, bug tracker            | 4 weeks  |
| 6. Deployment            | CI/CD setup, staging, production release                | Deployment runbook, monitoring setup | 2 weeks  |
| 7. Maintenance           | Bug fixes, feature enhancements, performance tuning     | Patch notes, updated documentation   | Ongoing  |

#### 3.2 Agile Sprint Plan (High Level)

Development follows a 2-week sprint cadence with the following sprint allocation:

- Sprints 1–2: Authentication, User Management, RBAC infrastructure
- Sprints 3–4: Brand Depot CRUD, Product Management, Assignments
- Sprints 5–6: Visit Tracking, GPS integration, Check-in/Check-out
- Sprints 7–8: Image Upload system, Activity Logs, Audit Trail
- Sprints 9–10: Dashboard, Reporting Engine, Export PDF/Excel
- Sprints 11–12: Real-time Socket.IO, Notification System, Push Notifications
- Sprints 13–14: Analytics AI, Search, Performance Optimization



## 4. Functional Requirements

### 4.1 Authentication & Authorization Module

| FR-ID      | Requirement                                                                       | Priority |
|------------|-----------------------------------------------------------------------------------|----------|
| FR-AUTH-01 | System shall authenticate users via username/password with JWT                    | Critical |
| FR-AUTH-02 | System shall issue short-lived access tokens (15 min) and refresh tokens (7 days) | Critical |
| FR-AUTH-03 | System shall implement role-based access control (RBAC)                           | Critical |
| FR-AUTH-04 | System shall enforce password hashing using bcrypt (cost factor $\geq 12$ )       | Critical |
| FR-AUTH-05 | System shall store and display login history per user                             | High     |
| FR-AUTH-06 | System shall support forced logout / session revocation                           | High     |
| FR-AUTH-07 | System shall support password reset via email OTP                                 | High     |
| FR-AUTH-08 | System shall lock accounts after 5 consecutive failed login attempts              | High     |

### 4.2 User Management Module

| FR-ID     | Requirement                                                               | Priority |
|-----------|---------------------------------------------------------------------------|----------|
| FR-USR-01 | Admin can create users with role assignment                               | Critical |
| FR-USR-02 | Admin can deactivate/reactivate user accounts                             | Critical |
| FR-USR-03 | Admin can assign and modify user roles and permissions                    | Critical |
| FR-USR-04 | System shall support four roles: Admin, Manager, Field Handler, Viewer    | Critical |
| FR-USR-05 | Each user has a profile with contact info, avatar, and linked assignments | Medium   |

### 4.3 Brand Depot Management Module

| FR-ID     | Requirement                                                    | Priority |
|-----------|----------------------------------------------------------------|----------|
| FR-DEP-01 | Authorized users can create, edit, delete (soft) depot records | Critical |
| FR-DEP-02 | Each depot has GPS coordinates (latitude/longitude)            | Critical |
| FR-DEP-03 | Depots can be filtered by Province, District, Status, Handler  | Critical |

| FR-ID     | Requirement                                                          | Priority |
|-----------|----------------------------------------------------------------------|----------|
| FR-DEP-04 | Each depot can be assigned one or more products                      | Critical |
| FR-DEP-05 | Each depot can be assigned a primary field handler                   | Critical |
| FR-DEP-06 | Depot status can be set to Active, Inactive, Suspended, Under Review | High     |
| FR-DEP-07 | Depot records store contact name, phone, address, and business type  | High     |
| FR-DEP-08 | Depot map view shows all depots on an interactive map                | High     |
| FR-DEP-09 | System supports bulk import of depots via CSV/Excel                  | Medium   |
| FR-DEP-10 | QR code generation for each depot for field handler check-in         | Medium   |

## 4.4 Product Management Module

| FR-ID     | Requirement                                                                        | Priority |
|-----------|------------------------------------------------------------------------------------|----------|
| FR-PRD-01 | Admin/Manager can create product categories and subcategories                      | Critical |
| FR-PRD-02 | Products have SKU, name, category, brand, and unit of measure                      | Critical |
| FR-PRD-03 | Products have an availability status per depot (In Stock, Low Stock, Out of Stock) | Critical |
| FR-PRD-04 | Product analytics show distribution by depot, region, and time period              | High     |
| FR-PRD-05 | Handlers can update product status during depot visits                             | High     |

## 4.5 Visit Tracking Module

| FR-ID     | Requirement                                                                       | Priority |
|-----------|-----------------------------------------------------------------------------------|----------|
| FR-VIS-01 | Handler performs GPS-verified check-in at depot location                          | Critical |
| FR-VIS-02 | System validates check-in GPS within configurable geofence radius (default: 100m) | Critical |
| FR-VIS-03 | Handler performs GPS-verified check-out to record visit duration                  | Critical |
| FR-VIS-04 | Handler can attach visit notes and observations                                   | Critical |
| FR-VIS-05 | Handler can upload multiple photos during visit (shelf, products, promotions)     | Critical |
| FR-VIS-06 | Visit history shows timeline of all depot visits                                  | High     |

| FR-ID     | Requirement                                                    | Priority |
|-----------|----------------------------------------------------------------|----------|
| FR-VIS-07 | System supports QR code scan as alternative to GPS check-in    | Medium   |
| FR-VIS-08 | Visit records capture device info and battery status for audit | Medium   |

## 4.6 Reporting & Analytics Module

| FR-ID     | Requirement                                                                | Priority |
|-----------|----------------------------------------------------------------------------|----------|
| FR-RPT-01 | System generates weekly and monthly operational reports automatically      | Critical |
| FR-RPT-02 | Reports are exportable as PDF and Excel (XLSX)                             | Critical |
| FR-RPT-03 | Handler performance reports include visit count, duration, compliance rate | Critical |
| FR-RPT-04 | District/Province analytics show depot coverage and product availability   | High     |
| FR-RPT-05 | Dashboard displays real-time KPI cards with trend indicators               | High     |
| FR-RPT-06 | Heatmap visualization shows geographic distribution of field activity      | Medium   |
| FR-RPT-07 | AI-powered analytics provide predictive insights and anomaly detection     | Low      |

## 5. Non-Functional Requirements

| NFR-ID | Category       | Requirement                          | Target         |
|--------|----------------|--------------------------------------|----------------|
| NFR-01 | Performance    | API response time (95th percentile)  | < 200ms        |
| NFR-02 | Performance    | Dashboard load time (initial)        | < 2 seconds    |
| NFR-03 | Performance    | Image upload and processing time     | < 5 seconds    |
| NFR-04 | Scalability    | Concurrent active users supported    | > 1,000        |
| NFR-05 | Scalability    | Daily API requests capacity          | > 1,000,000    |
| NFR-06 | Availability   | System uptime SLA                    | 99.9%          |
| NFR-07 | Availability   | Planned maintenance window           | < 2 hrs/month  |
| NFR-08 | Security       | JWT token expiry – access token      | 15 minutes     |
| NFR-09 | Security       | Password minimum complexity          | 8 chars, mixed |
| NFR-10 | Security       | API rate limiting per IP             | 100 req/15 min |
| NFR-11 | Data Integrity | Database transaction ACID compliance | Full           |
| NFR-12 | Backup         | Automated database backup frequency  | Daily          |
| NFR-13 | Recovery       | RTO (Recovery Time Objective)        | < 4 hours      |
| NFR-14 | Recovery       | RPO (Recovery Point Objective)       | < 1 hour       |
| NFR-15 | Mobile         | Offline data sync support            | Required       |
| NFR-16 | Mobile         | App startup time (cold)              | < 3 seconds    |
| NFR-17 | Observability  | System log retention                 | 90 days        |
| NFR-18 | Compliance     | GDPR/data privacy field masking      | Required       |

## 6. User Roles & Permissions

### 6.1 Role Definitions

| Role          | Description                                                    | Scope                      |
|---------------|----------------------------------------------------------------|----------------------------|
| Super Admin   | Platform-level administrator with unrestricted access          | All modules, all data      |
| Admin         | Company-level admin managing users, depots, and configuration  | Company scope, all modules |
| Manager       | Supervisory role overseeing assigned field handlers and depots | Assigned handlers/depots   |
| Field Handler | Operational staff performing depot visits and updates          | Assigned depots only       |
| Viewer        | Read-only stakeholder for dashboards and reports               | Reports and analytics only |

### 6.2 Permission Matrix

| Permission      | Super Admin | Admin | Manager  | Field Handler | Viewer |
|-----------------|-------------|-------|----------|---------------|--------|
| Create Users    | ✓           | ✓     | X        | X             | X      |
| Manage Roles    | ✓           | ✓     | X        | X             | X      |
| Create Depots   | ✓           | ✓     | ✓        | X             | X      |
| Edit Depots     | ✓           | ✓     | ✓        | X             | X      |
| Delete Depots   | ✓           | ✓     | X        | X             | X      |
| View Depots     | ✓           | ✓     | ✓        | Assigned only | ✓      |
| Perform Visit   | ✓           | ✓     | ✓        | ✓             | X      |
| Upload Images   | ✓           | ✓     | ✓        | ✓             | X      |
| View Reports    | ✓           | ✓     | ✓        | Own reports   | ✓      |
| Export Reports  | ✓           | ✓     | ✓        | X             | X      |
| System Settings | ✓           | ✓     | X        | X             | X      |
| View Audit Logs | ✓           | ✓     | Own logs | X             | X      |

## 7. User Stories

### 7.1 Field Handler Stories

| Story ID | User Story                                                                                                         | Acceptance Criteria                                                                                         |
|----------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| US-FH-01 | As a Field Handler, I want to check in to a depot using GPS so that my visit is verified.                          | GPS coordinates captured; system validates location within geofence; check-in record created with timestamp |
| US-FH-02 | As a Field Handler, I want to upload shelf photos during my visit so that managers can see current conditions.     | Multiple photos uploaded; photos linked to visit record; images stored in cloud                             |
| US-FH-03 | As a Field Handler, I want to update product availability status at a depot so that stock records are current.     | Product status updated with timestamp and handler ID; history maintained                                    |
| US-FH-04 | As a Field Handler, I want to view my assigned depots on a map so that I can plan my route efficiently.            | All assigned depots shown on map; filter by status; route planning overlay                                  |
| US-FH-05 | As a Field Handler, I want the app to work offline so that I can continue working in areas with poor connectivity. | Visits/photos stored locally; auto-syncs when connectivity restored; sync status indicator shown            |

### 7.2 Manager Stories

| Story ID | User Story                                                                                                      | Acceptance Criteria                                                            |
|----------|-----------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| US-MG-01 | As a Manager, I want to see a real-time map of active field handlers so that I can monitor field operations.    | Live handler locations shown; last update timestamp; status indicators         |
| US-MG-02 | As a Manager, I want to receive alerts when a handler misses a scheduled depot visit so that I can take action. | Alert sent via push notification and email; configurable thresholds            |
| US-MG-03 | As a Manager, I want to generate a monthly performance report for each handler so that I can evaluate KPIs.     | Report generated in PDF/Excel; includes visit count, duration, compliance rate |
| US-MG-04 | As a Manager, I want to assign depots to specific handlers so that accountability is clearly defined.           | Assignment recorded with date; visible on handler profile and depot record     |

### 7.3 Admin Stories

| Story ID | User Story                                                                                                        | Acceptance Criteria                                                        |
|----------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| US-AD-01 | As an Admin, I want to create and manage user accounts with specific roles so that access is properly controlled. | User created; role assigned; welcome email sent; login works immediately   |
| US-AD-02 | As an Admin, I want to import a list of depots from CSV so that onboarding is efficient.                          | CSV validated; records created; duplicate detection; error report provided |
| US-AD-03 | As an Admin, I want to view a full audit trail of all system activities so that security can be monitored.        | All CRUD actions logged; filterable by user, date, action type; exportable |

## 8. Database Design

### 8.1 Database Architecture Overview

The system uses PostgreSQL as the primary relational database with Prisma ORM for schema management and type-safe query building. The schema is designed following Third Normal Form (3NF) principles with strategic denormalization for reporting performance.

**Design Principles**

- Soft delete pattern: All core tables include deleted\_at TIMESTAMP NULL
- UUID primary keys: All tables use UUID v4 for horizontal scalability
- Audit columns: created\_at, updated\_at, created\_by, updated\_by on all tables
- Indexes: Composite indexes on frequently joined and filtered columns
- Constraints: FK constraints with appropriate cascade rules

### 8.2 Core Entity Relationship Description

The following describes the primary relationships in the data model:

- users (1) → (M) assignments: A user can be assigned to multiple depots over time
- brand\_depots (1) → (M) assignments: A depot can have multiple handler assignments (historical)
- brand\_depots (1) → (M) visits: A depot accumulates many visits over its lifetime
- visits (1) → (M) visit\_images: Each visit may include multiple uploaded photos
- brand\_depots (M) ↔ (M) products: Via depot\_products junction table
- users (1) → (M) activity\_logs: All user actions generate audit log entries
- visits (1) → (M) product\_status\_updates: Product statuses are updated within visits

### 8.3 PostgreSQL Schema

**Table: users**

```
CREATE TABLE users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  username    VARCHAR(100) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  first_name  VARCHAR(100) NOT NULL,  
  last_name   VARCHAR(100) NOT NULL,  
  phone       VARCHAR(20),  
  avatar_url  TEXT,  
  role        user_role_enum NOT NULL DEFAULT 'field_handler',  
  status      user_status_enum NOT NULL DEFAULT 'active',  
  last_login_at TIMESTAMP,  
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),  
);
```



```

    updated_at      TIMESTAMP NOT NULL DEFAULT NOW(),
    deleted_at      TIMESTAMP
);

CREATE TYPE user_role_enum AS ENUM
('super_admin','admin','manager','field_handler','viewer');
CREATE TYPE user_status_enum AS ENUM ('active','inactive','suspended');

CREATE INDEX idx_users_email ON users(email) WHERE deleted_at IS NULL;
CREATE INDEX idx_users_role ON users(role) WHERE deleted_at IS NULL;

```

## Table: brand\_depots

```

CREATE TABLE brand_depots (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(255) NOT NULL,
  code        VARCHAR(50) UNIQUE NOT NULL,
  address      TEXT NOT NULL,
  province     VARCHAR(100),
  district     VARCHAR(100),
  latitude     DECIMAL(10,8) NOT NULL,
  longitude     DECIMAL(11,8) NOT NULL,
  geofence_radius INT NOT NULL DEFAULT 100, -- meters
  contact_name VARCHAR(255),
  contact_phone VARCHAR(20),
  status       depot_status_enum NOT NULL DEFAULT 'active',
  qr_code_url  TEXT,
  created_by   UUID REFERENCES users(id),
  updated_by   UUID REFERENCES users(id),
  created_at   TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at   TIMESTAMP
);

CREATE TYPE depot_status_enum AS ENUM
('active','inactive','suspended','under_review');

CREATE INDEX idx_depots_province_district ON brand_depots(province, district);
CREATE INDEX idx_depots_status ON brand_depots(status) WHERE deleted_at IS NULL;
-- PostGIS spatial index for geo queries
CREATE INDEX idx_depots_location ON brand_depots USING GIST(
  ST_MakePoint(longitude, latitude)::geography
);

```

## Table: products

```

CREATE TABLE products (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sku         VARCHAR(100) UNIQUE NOT NULL,
  name        VARCHAR(255) NOT NULL,
  description  TEXT,

```

```

    category_id  UUID REFERENCES product_categories(id),
    brand        VARCHAR(255),
    unit         VARCHAR(50),
    image_url    TEXT,
    status       product_status_enum NOT NULL DEFAULT 'active',
    created_at   TIMESTAMP NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMP NOT NULL DEFAULT NOW(),
    deleted_at   TIMESTAMP
);

CREATE TABLE product_categories (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name         VARCHAR(255) NOT NULL,
    parent_id    UUID REFERENCES product_categories(id)
);

-- Junction table for depot <-> product M:M
CREATE TABLE depot_products (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    depot_id     UUID NOT NULL REFERENCES brand_depots(id) ON DELETE CASCADE,
    product_id   UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    availability availability_enum NOT NULL DEFAULT 'in_stock',
    updated_at   TIMESTAMP NOT NULL DEFAULT NOW(),
    UNIQUE(depot_id, product_id)
);

CREATE TYPE availability_enum AS ENUM ('in_stock','low_stock','out_of_stock');
```

## Table: visits

```

CREATE TABLE visits (
    id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    depot_id     UUID NOT NULL REFERENCES brand_depots(id),
    handler_id   UUID NOT NULL REFERENCES users(id),
    status       visit_status_enum NOT NULL DEFAULT 'in_progress',
    check_in_at  TIMESTAMP NOT NULL DEFAULT NOW(),
    check_in_lat DECIMAL(10,8) NOT NULL,
    check_in_lng DECIMAL(11,8) NOT NULL,
    check_in_method checkin_method_enum NOT NULL DEFAULT 'gps',
    check_out_at TIMESTAMP,
    check_out_lat DECIMAL(10,8),
    check_out_lng DECIMAL(11,8),
    duration_mins INT, -- computed on checkout
    notes        TEXT,
    is_geofence_valid BOOLEAN NOT NULL DEFAULT FALSE,
    device_info   JSONB,
    created_at   TIMESTAMP NOT NULL DEFAULT NOW()
);

CREATE TYPE visit_status_enum AS ENUM
('in_progress','completed','cancelled','flagged');
CREATE TYPE checkin_method_enum AS ENUM ('gps','qr_code','manual');
```

```
CREATE INDEX idx_visits_depot_id ON visits(depot_id);
CREATE INDEX idx_visits_handler_id ON visits(handler_id);
CREATE INDEX idx_visits_check_in_at ON visits(check_in_at DESC);
```

## Tables: assignments, visit\_images, activity\_logs, notifications

```
-- Handler-to-Depot Assignments
CREATE TABLE assignments (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  depot_id    UUID NOT NULL REFERENCES brand_depots(id),
  handler_id  UUID NOT NULL REFERENCES users(id),
  assigned_by UUID REFERENCES users(id),
  start_date  DATE NOT NULL DEFAULT CURRENT_DATE,
  end_date    DATE,
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

-- Visit Images
CREATE TABLE visit_images (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  visit_id    UUID NOT NULL REFERENCES visits(id) ON DELETE CASCADE,
  url         TEXT NOT NULL,
  type        image_type_enum NOT NULL DEFAULT 'shelf',
  caption     TEXT,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE TYPE image_type_enum AS ENUM
('shelf','product','promotion','damage','other');

-- Audit / Activity Logs
CREATE TABLE activity_logs (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID REFERENCES users(id),
  action      VARCHAR(100) NOT NULL, -- e.g. 'depot.update'
  entity_type VARCHAR(100),         -- e.g. 'brand_depot'
  entity_id   UUID,
  ip_address  INET,
  user_agent  TEXT,
  old_value   JSONB,
  new_value   JSONB,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_activity_logs_user ON activity_logs(user_id);
CREATE INDEX idx_activity_logs_entity ON activity_logs(entity_type, entity_id);
CREATE INDEX idx_activity_logs_created ON activity_logs(created_at DESC);

-- Notifications
CREATE TABLE notifications (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  type        VARCHAR(100) NOT NULL,
  title       VARCHAR(255) NOT NULL,
```

```
body          TEXT NOT NULL,  
data          JSONB,  
is_read       BOOLEAN NOT NULL DEFAULT FALSE,  
created_at    TIMESTAMP NOT NULL DEFAULT NOW()  
);  
CREATE INDEX idx_notifications_user_unread ON notifications(user_id, is_read)  
WHERE is_read = FALSE;
```

## 9. Backend Architecture

### 9.1 Architecture Pattern: Modular Monolith → Microservices

The system is designed as a Modular Monolith initially, with clear module boundaries enabling future extraction into microservices. This approach delivers the simplicity of a monolith during early development while avoiding the distributed system complexity before scale demands it.

### 9.2 Folder Structure

```
src/
├── config/           # App config, env variables, database, redis
├── modules/          # Feature modules (self-contained)
│   ├── auth/
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   ├── auth.repository.ts
│   │   ├── auth.routes.ts
│   │   └── auth.dto.ts
│   ├── users/
│   ├── depots/
│   ├── products/
│   ├── visits/
│   ├── images/
│   ├── reports/
│   └── notifications/
├── shared/
│   ├── middleware/  # auth, rbac, rateLimiter, logger
│   ├── utils/       # helpers, validators, transforms
│   ├── types/       # shared TypeScript types
│   └── errors/      # custom error classes
├── infrastructure/
│   ├── database/    # Prisma client, migrations
│   ├── redis/       # Redis client, cache helpers
│   ├── queue/       # RabbitMQ producer/consumer
│   ├── storage/     # S3/Cloudinary adapters
│   └── socket/      # Socket.IO setup
├── jobs/            # Cron jobs, background processors
├── app.ts           # Express app setup
└── server.ts        # Server entry point
```

### 9.3 API Versioning Strategy

All APIs are versioned under `/api/v1/` to enable backward-compatible evolution. Version negotiation is handled through URL path versioning, with a deprecation notice header added to legacy versions before sunseting.

### 9.4 Middleware Stack

| Middleware     | Library                      | Purpose                                     |
|----------------|------------------------------|---------------------------------------------|
| Helmet         | helmet                       | HTTP security headers (CSP, HSTS, X-Frame)  |
| CORS           | cors                         | Cross-origin resource sharing policy        |
| Rate Limiter   | express-rate-limit + Redis   | API abuse prevention, per-IP throttling     |
| Request Logger | morgan + winston             | Access logging with request ID              |
| Body Parser    | express.json()               | JSON request body parsing                   |
| Authentication | custom JWT middleware        | Token validation and user context injection |
| RBAC           | custom permission middleware | Role and permission enforcement             |
| Validation     | zod schema middleware        | Request body and parameter validation       |
| Error Handler  | custom global handler        | Centralized error formatting and logging    |
| Compression    | compression                  | Gzip response compression                   |

### 9.5 Centralized Response Format

```
// Success Response
{
  "success": true,
  "statusCode": 200,
  "message": "Depots retrieved successfully",
  "data": { ... },
  "meta": {
    "page": 1, "limit": 20, "total": 150, "totalPages": 8
  },
  "requestId": "req_abc123"
}

// Error Response
{
  "success": false,
  "statusCode": 422,
  "message": "Validation failed",
  "errors": [
    { "field": "latitude", "message": "Must be between -90 and 90" }
  ],
  "requestId": "req_abc123"
}
```

## 10. REST API Design

### 10.1 Authentication Endpoints

| Method | Endpoint                     | Description                    | Auth Required |
|--------|------------------------------|--------------------------------|---------------|
| POST   | /api/v1/auth/login           | User login, returns JWT tokens | No            |
| POST   | /api/v1/auth/refresh         | Refresh access token           | Refresh token |
| POST   | /api/v1/auth/logout          | Revoke refresh token           | Yes           |
| POST   | /api/v1/auth/forgot-password | Send password reset OTP        | No            |
| POST   | /api/v1/auth/reset-password  | Reset password with OTP        | No            |
| GET    | /api/v1/auth/me              | Get current user profile       | Yes           |

### 10.2 Depot Endpoints

| Method | Endpoint                    | Description                         | Auth Required  |
|--------|-----------------------------|-------------------------------------|----------------|
| GET    | /api/v1/depots              | List depots (paginated, filterable) | Yes            |
| POST   | /api/v1/depots              | Create new depot                    | Admin, Manager |
| GET    | /api/v1/depots/:id          | Get depot details                   | Yes            |
| PUT    | /api/v1/depots/:id          | Update depot                        | Admin, Manager |
| DELETE | /api/v1/depots/:id          | Soft delete depot                   | Admin          |
| GET    | /api/v1/depots/:id/visits   | Get depot visit history             | Yes            |
| GET    | /api/v1/depots/:id/products | Get depot products                  | Yes            |
| POST   | /api/v1/depots/:id/assign   | Assign handler to depot             | Admin, Manager |
| GET    | /api/v1/depots/map          | Get all depots as GeoJSON for map   | Yes            |
| GET    | /api/v1/depots/:id/qr-code  | Generate/retrieve depot QR code     | Yes            |

### 10.3 Visit Endpoints

| Method | Endpoint                     | Description              | Auth Required |
|--------|------------------------------|--------------------------|---------------|
| POST   | /api/v1/visits/check-in      | GPS check-in to depot    | Field Handler |
| POST   | /api/v1/visits/:id/check-out | GPS check-out from depot | Field Handler |

| Method | Endpoint                  | Description              | Auth Required |
|--------|---------------------------|--------------------------|---------------|
| GET    | /api/v1/visits            | List visits (filterable) | Yes           |
| GET    | /api/v1/visits/:id        | Get visit detail         | Yes           |
| PUT    | /api/v1/visits/:id/notes  | Update visit notes       | Field Handler |
| POST   | /api/v1/visits/:id/images | Upload visit images      | Field Handler |
| GET    | /api/v1/visits/:id/images | Get visit images         | Yes           |

## 10.4 Reports Endpoints

| Method | Endpoint                     | Description                     | Auth Required |
|--------|------------------------------|---------------------------------|---------------|
| GET    | /api/v1/reports/dashboard    | KPI summary for dashboard       | Yes           |
| GET    | /api/v1/reports/handler/:id  | Handler performance report      | Manager+      |
| GET    | /api/v1/reports/district     | District-level analytics        | Manager+      |
| GET    | /api/v1/reports/products     | Product availability analytics  | Manager+      |
| GET    | /api/v1/reports/export/pdf   | Export report as PDF            | Manager+      |
| GET    | /api/v1/reports/export/excel | Export report as Excel          | Manager+      |
| GET    | /api/v1/reports/heatmap      | Heatmap data by geo coordinates | Manager+      |



# 11. Frontend Architecture

## 11.1 React.js Project Structure

```
src/
├── api/                # Axios instances, API service functions
│   ├── client.ts       # Axios base config with interceptors
│   ├── auth.api.ts
│   ├── depots.api.ts
│   └── visits.api.ts
├── app/                # Router, providers, layout
├── components/         # Reusable UI components
│   ├── ui/            # Base components (Button, Input, Badge)
│   ├── forms/         # Form components with RHF
│   ├── tables/        # DataTable, Pagination
│   ├── charts/        # Chart wrappers (Recharts)
│   └── maps/          # Leaflet/Google Maps wrappers
├── features/           # Feature-specific components
│   ├── auth/
│   ├── dashboard/
│   ├── depots/
│   ├── visits/
│   ├── reports/
│   └── users/
├── hooks/              # Custom React hooks
├── store/              # Redux Toolkit slices
├── utils/              # Formatters, validators, constants
└── types/              # TypeScript interfaces
```

## 11.2 State Management Strategy

| State Type      | Solution                     | Use Cases                                   |
|-----------------|------------------------------|---------------------------------------------|
| Server State    | React Query (TanStack Query) | API data fetching, caching, synchronization |
| Global UI State | Redux Toolkit                | Auth user, notifications, sidebar state     |
| Form State      | React Hook Form + Zod        | Form validation, submission, error handling |
| Real-time State | Socket.IO + Redux            | Live activity feed, online status, alerts   |
| URL State       | React Router + query-string  | Filters, pagination, active tabs            |

## 11.3 Key Frontend Libraries

| Library          | Version | Purpose                               |
|------------------|---------|---------------------------------------|
| react-router-dom | v6      | Client-side routing, protected routes |

| Library                 | Version | Purpose                               |
|-------------------------|---------|---------------------------------------|
| @tanstack/react-query   | v5      | Server state management, caching      |
| @reduxjs/toolkit        | latest  | Global client state                   |
| axios                   | latest  | HTTP client with interceptors         |
| react-hook-form         | v7      | Performant form management            |
| zod                     | latest  | Runtime schema validation             |
| recharts                | latest  | Charts and data visualization         |
| leaflet / react-leaflet | latest  | Interactive map visualization         |
| react-table (TanStack)  | v8      | Headless table with sorting/filtering |
| socket.io-client        | latest  | Real-time WebSocket connection        |
| react-hot-toast         | latest  | Toast notifications                   |
| date-fns                | latest  | Date formatting and manipulation      |

## 12. Mobile Architecture

### 12.1 Framework Selection

React Native is recommended as the mobile framework given the existing React.js web codebase, enabling code sharing of business logic, API services, validation schemas, and TypeScript types between web and mobile.

| Capability         | Library/Approach                 | Notes                                  |
|--------------------|----------------------------------|----------------------------------------|
| Navigation         | React Navigation v6              | Stack + Tab + Drawer navigators        |
| HTTP Client        | Axios + React Query              | Shared API layer with web              |
| State              | Redux Toolkit                    | Shared slice logic with web            |
| GPS Location       | react-native-geolocation-service | Background location tracking           |
| Camera/Photos      | react-native-image-picker        | Photo capture during visits            |
| QR Scanner         | react-native-vision-camera       | Depot QR code check-in                 |
| Offline DB         | WatermelonDB / SQLite            | Local data persistence for offline use |
| Push Notifications | Firebase Cloud Messaging (FCM)   | Cross-platform push                    |
| Maps               | react-native-maps                | Depot map view on mobile               |
| Background Sync    | react-native-background-task     | Sync offline data on connectivity      |
| Biometrics         | react-native-biometrics          | Fingerprint/Face ID login              |

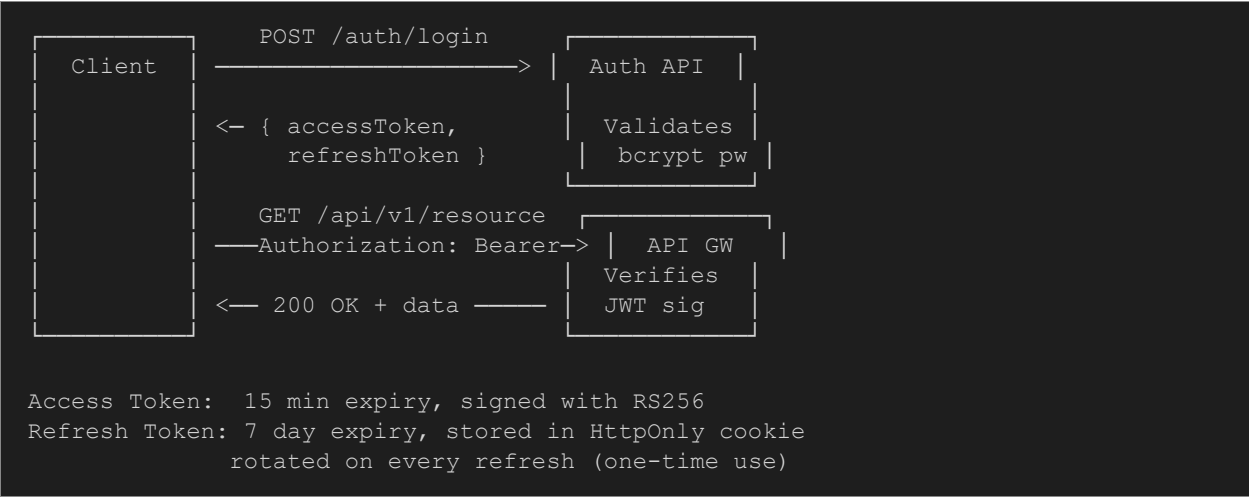
### 12.2 Offline Sync Strategy

The mobile app implements a local-first architecture for field operations:

1. Field handler initiates check-in or visit action on mobile
2. Action is persisted immediately to local WatermelonDB/SQLite
3. Optimistic UI update shown to user (no loading spinner)
4. Background sync job attempts to push to server API
5. On success: local record marked as synced; conflict resolution applied
6. On failure: retry with exponential backoff (max 24 hours retention)
7. Sync status icon visible in app header (synced / pending / error)

# 13. Security Architecture

## 13.1 JWT Authentication Flow



## 13.2 Security Controls

| Threat               | Control                                      | Implementation                            |
|----------------------|----------------------------------------------|-------------------------------------------|
| Brute force login    | Account lockout + rate limiting              | 5 failures = 30min lock; 100 req/15min/IP |
| SQL injection        | Parameterized queries via Prisma ORM         | ORM prevents direct SQL construction      |
| XSS                  | Content Security Policy + input sanitization | Helmet.js CSP headers; DOMPurify on FE    |
| CSRF                 | SameSite cookie + CSRF token                 | HttpOnly SameSite=Strict cookie           |
| JWT token theft      | Short expiry + HttpOnly refresh cookie       | 15min access; RT in HttpOnly cookie       |
| Privilege escalation | RBAC middleware on every route               | Permission check before controller        |
| Mass assignment      | DTO whitelist via Zod schemas                | Only whitelisted fields accepted          |
| Insecure file upload | File type validation + virus scan            | Mime type check; max 10MB; S3 scan        |
| Exposed secrets      | Environment variables + AWS Secrets Manager  | No secrets in codebase                    |

## 13.3 Password Security

```
// bcrypt with cost factor 12
const hash = await bcrypt.hash(plainPassword, 12);

// Password policy enforced via Zod:
const passwordSchema = z.string()
  .min(8, "Minimum 8 characters")
  .regex(/[A-Z]/, "At least one uppercase letter")
  .regex(/[0-9]/, "At least one number")
  .regex(/^[A-Za-z0-9]/, "At least one special character");
```

## 14. DevOps & Deployment Architecture

### 14.1 Docker Compose (Development)

```
version: '3.9'
services:
  api:
    build: ./backend
    ports: ['3000:3000']
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/bdmfots
      REDIS_URL: redis://redis:6379
    depends_on: [db, redis]

  frontend:
    build: ./frontend
    ports: ['5173:5173']

  db:
    image: postgis/postgis:15-3.3
    volumes: ['pgdata:/var/lib/postgresql/data']

  redis:
    image: redis:7-alpine
    volumes: ['redisdata:/data']

  nginx:
    image: nginx:alpine
    ports: ['80:80', '443:443']
    volumes: ['./nginx.conf:/etc/nginx/nginx.conf']
```

### 14.2 GitHub Actions CI/CD Pipeline

```
# .github/workflows/deploy.yml
on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install & Test
        run: npm ci && npm test

  build-and-push:
    needs: test
    steps:
```

```
- name: Build Docker image
  run: docker build -t $ECR_REGISTRY/bdmfots-api:${{ github.sha }} .
- name: Push to ECR
  run: docker push $ECR_REGISTRY/bdmfots-api:${{ github.sha }}

deploy:
  needs: build-and-push
  steps:
    - name: Deploy to ECS / Kubernetes
      run: aws ecs update-service --service bdmfots-api --force-new-deployment
```

14.3 Infrastructure Architecture

| Component     | Service                                     | Purpose                              |
|---------------|---------------------------------------------|--------------------------------------|
| Load Balancer | AWS ALB / Nginx                             | Distribute traffic, SSL termination  |
| API Servers   | EC2 Auto Scaling / ECS Fargate              | Stateless API instances (2–10 nodes) |
| Database      | AWS RDS PostgreSQL Multi-AZ                 | Primary + standby failover           |
| Cache         | AWS ElastiCache Redis Cluster               | Session, API response caching        |
| File Storage  | AWS S3 + CloudFront CDN                     | Images and static assets             |
| Queue         | AWS SQS / RabbitMQ on ECS                   | Async job processing                 |
| Monitoring    | CloudWatch + Grafana                        | Metrics, dashboards, alerting        |
| Logging       | ELK Stack (ElasticSearch, Logstash, Kibana) | Centralized log management           |
| Secrets       | AWS Secrets Manager                         | Secure credential storage            |
| DNS           | Route 53                                    | Domain management, health routing    |

## 15. Real-Time & Queue Architecture

### 15.1 Socket.IO Events

| Event Name       | Direction       | Payload                            | Consumer                |
|------------------|-----------------|------------------------------------|-------------------------|
| handler:location | Client → Server | { handlerId, lat, lng, timestamp } | Manager dashboard       |
| visit:checkin    | Server → Client | { visitId, depotId, handlerName }  | Manager clients in room |
| visit:checkout   | Server → Client | { visitId, duration }              | Manager clients         |
| notification:new | Server → Client | { type, title, body }              | Targeted user           |
| depot:updated    | Server → Client | { depotId, changes }               | All subscribed users    |
| sync:status      | Server → Client | { pendingCount, lastSync }         | Mobile app              |

### 15.2 Queue Job Types

| Job Name                | Trigger                 | Description                                | Retry           |
|-------------------------|-------------------------|--------------------------------------------|-----------------|
| send_push_notification  | Visit check-in / alert  | Sends FCM push to mobile devices           | 3x exp. backoff |
| send_email_notification | Report ready / alert    | Sends transactional email via SES/SendGrid | 3x              |
| generate_report         | Scheduled cron / manual | Builds PDF/Excel report files              | 2x              |
| process_image           | Image upload            | Resize, optimize, generate thumbnails      | 2x              |
| sync_offline_data       | Mobile reconnect        | Reconcile offline visit records            | 5x              |
| run_geofence_check      | GPS coordinate received | Validate handler is within depot radius    | 1x              |



## 16. Caching Strategy

### 16.1 Redis Cache Layers

| Cache Key Pattern         | TTL    | Content                    | Invalidation Trigger     |
|---------------------------|--------|----------------------------|--------------------------|
| auth:refresh:{userId}     | 7 days | Refresh token metadata     | Logout / password change |
| session:{userId}          | 15 min | User session + permissions | Role change / logout     |
| depot:list:{filters_hash} | 5 min  | Paginated depot list       | Any depot CRUD           |
| depot:{id}                | 10 min | Single depot detail        | Depot update             |
| report:dashboard:{org}    | 2 min  | KPI summary stats          | Any visit event          |
| geocode:{lat}:{lng}       | 24 hrs | Reverse geocode result     | Never (immutable)        |
| user:profile:{id}         | 30 min | User profile data          | Profile update           |

# 17. Testing Strategy

## 17.1 Testing Pyramid

| Level             | Tool                  | Coverage Target         | What is Tested                              |
|-------------------|-----------------------|-------------------------|---------------------------------------------|
| Unit Tests        | Jest + ts-jest        | > 80%                   | Services, utils, validators, business logic |
| Integration Tests | Jest + Supertest      | > 70%                   | API endpoints, DB interactions, auth flows  |
| E2E Tests         | Playwright / Cypress  | > 50% of critical paths | Full user flows: login, visit, report       |
| Mobile E2E        | Detox                 | Core user flows         | Check-in, offline sync, photo upload        |
| Load Testing      | k6 / Artillery        | 1,000 concurrent users  | API throughput under sustained load         |
| Security Testing  | OWASP ZAP + npm audit | OWASP Top 10            | Vulnerability scanning                      |

## 18. Scalability Strategy

### 18.1 Horizontal Scaling Approach

The API is designed as a stateless service — all session state is stored in Redis, enabling multiple API instances behind a load balancer without sticky sessions. Auto Scaling Groups (AWS) or Kubernetes Horizontal Pod Autoscaler respond to CPU/memory thresholds.

### 18.2 Database Scaling

- Read replicas: Reporting and analytics queries routed to read replicas
- Connection pooling: PgBouncer manages DB connection limits
- Partitioning: visits and activity\_logs tables partitioned by month
- Archiving: Records older than 2 years moved to cold storage / data warehouse

### 18.3 Modular Monolith to Microservices Evolution

| Phase                  | Architecture                              | Trigger Condition                      |
|------------------------|-------------------------------------------|----------------------------------------|
| Phase 1 (0–6 months)   | Modular Monolith                          | < 5,000 depots, < 100 concurrent users |
| Phase 2 (6–18 months)  | Extract Reports Service                   | Reports generation causes API latency  |
| Phase 3 (18–36 months) | Extract Notifications + Location Services | > 500 concurrent field handlers        |
| Phase 4 (36+ months)   | Full Microservices + Service Mesh         | Multi-tenant SaaS expansion            |

## 19. Reporting & Analytics

### 19.1 Built-in Reports

| Report Name          | Frequency        | Metrics Included                                           | Format     |
|----------------------|------------------|------------------------------------------------------------|------------|
| Handler Performance  | Weekly / Monthly | Visit count, duration avg, compliance %, photo upload rate | PDF, Excel |
| Depot Coverage       | Weekly           | Visited vs. unvisited, coverage gap map                    | PDF, Excel |
| Product Availability | Daily / Weekly   | In stock %, low stock alerts, out-of-stock trends          | PDF, Excel |
| District Summary     | Monthly          | Depot count, visit frequency, handler allocation           | PDF, Excel |
| Activity Audit       | On demand        | All system actions with user attribution                   | Excel, CSV |
| System Health        | Automated        | API response times, error rates, sync failures             | Dashboard  |

## 20. Risk Analysis

| Risk                             | Probability | Impact   | Mitigation                                                                          |
|----------------------------------|-------------|----------|-------------------------------------------------------------------------------------|
| GPS spoofing by handlers         | Medium      | High     | Cross-validate GPS with accelerometer; anomaly detection; random photo verification |
| Database performance degradation | Medium      | High     | Partitioning, read replicas, query optimization, connection pooling                 |
| Mobile offline sync conflicts    | High        | Medium   | Last-write-wins with server timestamp; conflict log for manual review               |
| Image storage cost overrun       | Medium      | Medium   | Compression pipeline; lifecycle policies; tiered storage (S3 Glacier)               |
| Third-party GPS/Map API downtime | Low         | Medium   | Fallback to cached coordinates; OpenStreetMap as secondary provider                 |
| Unauthorized data access         | Low         | Critical | RBAC, encryption at rest and in transit, audit logging, MFA rollout                 |
| High staff onboarding friction   | Medium      | Medium   | Progressive onboarding, offline-first UX, in-app tutorial                           |

## 21. Future Enhancement Plan

---

### 21.1 Short-Term (0–6 Months Post-Launch)

- Route optimization: AI-powered daily route planning for field handlers
- WhatsApp / Telegram notifications as alternative to email
- Manager mobile app with live tracking dashboard
- Barcode scanning for product verification during visits

### 21.2 Medium-Term (6–18 Months)

- AI-powered shelf recognition: Auto-detect products from visit photos
- Predictive stock-out alerts using ML models on historical data
- Customer portal: Depot owners can view their own visit history
- Multi-language support (i18n framework integration)
- Multi-tenant architecture for SaaS expansion to multiple brand clients

### 21.3 Long-Term (18+ Months)

- IoT integration: Smart shelf sensors feeding real-time stock data
- Computer vision for planogram compliance checking
- Blockchain-based immutable audit trail for regulatory compliance
- Data marketplace: Anonymized brand distribution insights as a service

## 22. Final Recommendations

**Architecture Recommendation**

Start with Modular Monolith. Do NOT prematurely optimize to microservices.

The modular structure enables easy extraction later when actual scale demands it.

Focus engineering effort on data model quality and API contract stability.

**Technology Recommendations**

Use Prisma ORM over raw SQL for type safety and schema migration management.

Use React Query for all server state — it eliminates 70% of custom caching/loading code.

Use React Native (not Flutter) to maximize code sharing with the React web app.

Use Redis for rate limiting and session invalidation from day one.

**Process Recommendations**

Implement CI/CD from sprint 1 — deploy to staging on every merge to main.

Write integration tests before frontend — they serve as living API documentation.

Conduct weekly geofencing accuracy reviews with field handlers during rollout.

Plan a phased mobile rollout: web MVP first, mobile in sprints 13–14.

### 22.1 Development Priority Matrix

| Priority      | Module                        | Rationale                        |
|---------------|-------------------------------|----------------------------------|
| P0 – Critical | Auth + User Management + RBAC | Foundation for all other modules |
| P0 – Critical | Brand Depot CRUD + GPS        | Core business entity             |
| P1 – High     | Visit Tracking + Check-in/out | Primary accountability mechanism |
| P1 – High     | Image Upload + Activity Logs  | Evidence and audit requirements  |
| P2 – Medium   | Reporting Engine + Dashboard  | Business intelligence layer      |
| P2 – Medium   | Real-time Notifications       | Operational alerting             |
| P3 – Normal   | Mobile App + Offline Sync     | Field handler productivity       |
| P3 – Normal   | Advanced Analytics + Heatmaps | Strategic insights               |

*This document represents the complete enterprise system analysis for the Brand Depot Management & Field Operation Tracking System. All architectural decisions are grounded in production-proven patterns and designed for long-term maintainability, security, and scalability.*